

1. P-155, Ex-1:

```
# include <iostream>
using namespace std;
class coord{
int Fa,him;
public :
coord(int i,int k){Fa=i; him=k;} //setting value.
coord () { Fa=0; him=0; } //setting default value as 0.
coord operator*(coord &obj) {
    coord tem;
    tem.Fa= Fa*obj.Fa;
    tem.him =him*obj.him; //Overloading "*" operator so that o1 multiplied by o2 .
    return tem;}
coord operator/(coord &obj) {
    coord tem;
    tem.Fa= Fa/obj.Fa;
    tem.him =him/obj.him; //Overloading "/" operator so that o1 is divided by o2.
    return tem;}
void display(){cout<<"coord(x: "<<Fa<<" , y: "<<him<<)"<<endl;}
};
int main(){
coord a(10,10), b(5,2), c;
cout << "Object a: ";a.display();
cout << "Object b: ";b.display();
c = a * b;
cout << "After c = a * b: ";c.display();
c = a / b;
cout << "After c = a / b: ";c.display();
return 0;}
```

Output:

```
Object a: coord(x: 10, y: 10)
Object b: coord(x: 5, y: 2)
After c = a * b: coord(x: 50, y: 20)
After c = a / b: coord(x: 2, y: 5)
```

2. P-157, Ex-1:

```
# include <iostream>
using namespace std;
class coord{
int Fa,him;
public :
coord(int i,int k){Fa=i; him=k;} //setting value.
coord () { Fa=0; him=0; } //setting default value as 0.
bool operator>(coord ob2) {
```

```

return (Fa > ob2.Fa && him > ob2.him);} // Overload ">" operator if x & y in o1 is greater
than o2 then returns true.
bool operator<(coord ob2) {
return (Fa < ob2.Fa && him < ob2.him);} // Overload "<" operator if x & y in o1 is less
than o2 then returns true.
void display(){cout<<"coord(x: "<<Fa<<", y: "<<him<<)"<<endl;}
};
int main(){
coord o1(10,10), o2(5,2);
cout << "Object a: ";o1.display();
cout << "Object b: ";o2.display();
if (o1 > o2) {
    cout << "o1 is greater than o2\n";
} else {
    cout << "o1 is not greater than o2\n";
}
if (o1 < o2) {
    cout << "o1 is less than o2\n";
} else {
    cout << "o1 is not less than o2\n";
}
return 0;}

```

Output:

```

Object a: coord(x: 10, y: 10)
Object b: coord(x: 5, y: 2)
o1 is greater than o2
o1 is not less than o2

```

3. P-159, Ex-1:

```

#include <iostream>
using namespace std;
class coord{
int Fa,him;
public :
coord(int i,int k){Fa=i; him=k;} //setting value.
coord (){ Fa=0; him=0; } //setting default value as 0.
coord operator-() {Fa=-Fa;him=-him;
return *this;} // Prefix - operator overloading (Unary negation)
coord operator--(int){
    coord temp(Fa, him);
    Fa = -Fa;
    him = -him;
    return temp;} // Postfix - operator overloading (Unary negation)
void display(){cout<<"coord(x: "<<Fa<<", y: "<<him<<)"<<endl;}

```

```

};
int main(){
coord o1(10,10), o2,o3;
cout << "Original point: ";o1.display();
o2=-o1;
cout << "After prefix -: ";o2.display();
o3=o1--;
cout << "After postfix -: ";o3.display();
return 0;}

```

Output:

Original point: coord(x: 10, y: 10)
After prefix -: coord(x: -10, y: -10)
After postfix -: coord(x: -10, y: -10)

4. P-163, Ex-1:

```

# include <iostream >
using namespace std;
class coord{
int Fa,him;
public :
coord(int i,int k){Fa=i; him=k;} //setting value.
coord (){ Fa=0; him=0; } //setting default value as 0.
coord operator/(coord &obj) {
coord tem;tem.Fa= Fa/obj.Fa;
tem.him =him/obj.him; //Overloading "/" operator so that o1 is divided by o2.
return tem;}
coord operator-(coord &obj) {
coord tem;
tem.Fa= Fa-obj.Fa;
tem.him =him-obj.him; //Overloading "-" operator so that o1 is subtracted by o2.
return tem;}
void display(){cout<<"coord(x: "<<Fa<<" , y: "<<him<<)"<<endl;}
};
int main(){
coord o1(10,10), o2(5,2),o3;
cout << "Original point: ";o1.display();
cout << "Original point: ";o2.display();
o3 = o1 / o2;
cout << "After c = a / b: ";o3.display();
o3 = o1 - o2;
cout << "After c = a - b: ";o3.display();
return 0;}

```

Output:

Original point: coord(x: 10, y: 10)

After prefix -: coord(x: -10, y: -10)

After postfix -: coord(x: -10, y: -10)

5. P-163, Ex-2:

```
# include <iostream>
using namespace std;
class coord{
int Fa,him;
public :
coord(double i,double k){Fa=i; him=k;} //setting value.
coord (){ Fa=0; him=0; } //setting default value as 0.
friend coord operator*(const coord& c, int val){
return {c.Fa * val, c.him * val};} // Overload coord * int
friend coord operator*(int val, const coord& c){
return {c.Fa * val, c.him * val};} // Overload int * coord
void display(){cout<<"coord(x: "<<Fa<<", y: "<<him<<)"<<endl;}
};
int main(){
coord o1(10,10),o3;
cout << "Original point: ";o1.display();
o3 = o1 * 2;
cout << "After c = a - b: ";o3.display();
o3 = 3 * o1;
cout << "After c = a / b: ";o3.display();
return 0;}
```

Output:

Original point: coord(x: 10, y: 10)

After c = a - b: coord(x: 20, y: 20)

After c = a / b: coord(x: 30, y: 30)

6. P-163, Ex-4:

```
# include <iostream>
using namespace std;
class coord{
int Fa,him;
public :
coord(int i,int k){Fa=i; him=k;} //setting value.
coord (){ Fa=0; him=0; } //setting default value as 0.
friend coord& operator++(coord& c){++c.Fa;++c.him; // Overload prefix ++
return c;}
friend coord operator++(coord& c, int) { // Overload postfix ++
coord temp = c;c.Fa++;c.him++; // Save current state
```

```

return temp;} // Return saved state
void display(){cout<<"coord(x: "<<Fa<<", y: "<<him<<)"<<endl;}
};
int main(){
coord o1(10,10);
cout << "Original point: ";o1.display();
cout << "Prefix increment: ";(++o1).display();
cout << "Postfix increment: ";(o1++).display();
cout << "After postfix increment: ";o1.display();
return 0;}

```

Output:

Original point: coord(x: 10, y: 10)
Prefix increment: coord(x: 11, y: 11)
Postfix increment: coord(x: 11, y: 11)
After postfix increment: coord(x: 12, y: 12)

7. P-170, Ex-1:

```

#include <iostream>
using namespace std;
class Coord{
int Fa,him;
public :
Coord(int x_val=0,int y_val=0){Fa=x_val; him=y_val;} //setting Default value as 0.
Coord& operator<<(int shift) {
Fa= Fa-shift; him =him-shift; //Overloading "<<" operator so that both x & y are subtracted
//by given value
return *this;}
Coord& operator>>(int shift) {
Fa = Fa+shift; him =him+shift; //Overloading ">>" operator so that both x & y added by
//given value
return *this;}
void display(){cout<<"Coord(x: "<<Fa<<", y: "<<him<<)"<<endl;}
};
int main(){
Coord ob(10, 20);
cout << "Initial state: "; ob.display();
ob << 5; //subtract 5 from ob coordinates.
cout << "After ob << 5: "; ob.display();
ob >> 10; //added 10 to ob coordinates.
cout << "After ob >> 10: "; ob.display();
return 0;}

```

Output:

Original point: coord(x: 10, y: 10)

Original point: coord(x: 5, y: 2)

After c = a - b: coord(x: 5, y: 8)

After c = a / b: coord(x: 2, y: 5)

8. P-170, Ex-2:

```
# include <iostream>
using namespace std;
class three_d{
int Fa,h,im;
public :
three_d(int i,int j,int k){Fa=i; h=j; im=k;} //setting value.
three_d () { Fa=0; h=0; im=0; } //setting default value as 0.
three_d operator+(three_d &obj) {
    three_d tem;
    tem.Fa= Fa+obj.Fa;
    tem.h=h+obj.h;
    tem.im =im+obj.im; //Overloading "+" operator so that o1 & o2 summed.
    return tem;}
three_d operator-(three_d &obj) {
    three_d tem;
    tem.Fa= Fa-obj.Fa;
    tem.h=h-obj.h;
    tem.im =im-obj.im; //Overloading "-" operator so that o1 is subtracted by o2.
    return tem;}
three_d& operator++() {
    ++Fa; ++h; ++im; //Overloading "++" operator so that both x,y,z are increased by 1.
    return *this;}
three_d& operator--() {
    --Fa; --h; --im; //Overloading "--" operator so that both x,y,z are decreased by 1.
    return *this;}
void display(){cout<<"three_d(x: "<<Fa<<", y: "<<h<<", z: "<<im<<)"<<endl;}
};
int main(){
three_d a(1, 2, 3), b(4, 5, 6), c;
cout << "Object a: ";a.display();
cout << "Object b: ";b.display();
c = a + b;
cout << "After c = a + b: ";c.display();
c = a - b;
cout << "After c = a - b: ";c.display();
++a;
cout << "After ++a: ";a.display();
--b;
cout << "After --b: ";b.display();
return 0;}
```

Output:

Object a: three_d(x: 1, y: 2, z: 3)
Object b: three_d(x: 4, y: 5, z: 6)
After c = a + b: three_d(x: 5, y: 7, z: 9)
After c = a - b: three_d(x: -3, y: -3, z: -3)
After ++a: three_d(x: 2, y: 3, z: 4)
After --b: three_d(x: 3, y: 4, z: 5)

9. P-170, Ex-3:

```
# include <iostream >
using namespace std;
class three_d{
int Fa,h,im;
public :
three_d(int i,int j,int k){Fa=i; h=j; im=k;} //setting value.
three_d () { Fa=0; h=0; im=0; } //setting default value as 0.
three_d operator+( three_d &obj){ // Overloading "+" operator (takes reference parameter)
return three_d(Fa + obj.Fa, h + obj.h, im + obj.im);}
three_d operator-( three_d &obj){ // Overloading "-" operator (takes reference parameter)
return three_d(Fa - obj.Fa, h - obj.h, im - obj.im);}
friend three_d &operator++(three_d &obj) {
++obj.Fa; ++obj.h; ++obj.im; // Friend function to overload "++" (prefix increment)
return obj;}
friend three_d &operator--(three_d &obj) {
--obj.Fa; --obj.h; --obj.im; // Friend function to overload "--" (prefix decrement)
return obj;}
void display(){cout<<"three_d(x: "<<Fa<<", y: "<<h<<", z: "<<im<<)"<<endl;}
};
int main(){
three_d a(1, 2, 3), b(4, 5, 6), c;
cout << "Object a: ";a.display();
cout << "Object b: ";b.display();
c = a + b;
cout << "After c = a + b: ";c.display();
c = a - b;
cout << "After c = a - b: ";c.display();
++a;
cout << "After ++a: ";a.display();
--b;
cout << "After --b: ";b.display();
return 0;}
```

Output:

Object a: three_d(x: 1, y: 2, z: 3)
Object b: three_d(x: 4, y: 5, z: 6)
After c = a + b: three_d(x: 5, y: 7, z: 9)
After c = a - b: three_d(x: -3, y: -3, z: -3)
After ++a: three_d(x: 2, y: 3, z: 4)
After --b: three_d(x: 3, y: 4, z: 5)

10. P-170, Ex-7:

```
# include <iostream>
using namespace std;
class three_d{
int Fa,h,im;
public :
three_d(int i,int j,int k){Fa=i; h=j; im=k;} //setting value.
three_d () { Fa=0; h=0; im=0; } //setting default value as 0.
three_d operator+( three_d &obj){ // Overloading "+" operator (takes reference parameter)
return three_d(Fa + obj.Fa, h + obj.h, im + obj.im);}
friend three_d operator+(const three_d& obj, int val) { // Overload "+" operator for three_d +
int
return three_d(obj.Fa + val, obj.h + val, obj.im + val);}
three_d operator-( three_d &obj){ // Overloading "-" operator (takes reference parameter)
return three_d(Fa - obj.Fa, h - obj.h, im - obj.im);}
friend three_d operator+(int val, const three_d& obj) { // Overload "+" operator for int +
three_d
return three_d(obj.Fa + val, obj.h + val, obj.im + val);}
friend three_d &operator++(three_d &obj) {
++obj.Fa; ++obj.h; ++obj.im; // Friend function to overload "++" (prefix increment)
return obj;}
friend three_d &operator--(three_d &obj) {
--obj.Fa; --obj.h; --obj.im; // Friend function to overload "--" (prefix decrement)
return obj;}
void display(){cout<<"three_d(x: "<<Fa<<", y: "<<h<<", z: "<<im<<)"<<endl;}
};
int main(){
three_d a(1, 2, 3), b(4, 5, 6), c;
cout << "Object a: ";a.display();
cout << "Object b: ";b.display();
c = a + b;
cout << "After c = a + b: ";c.display();
c = a - b;
cout << "After c = a - b: ";c.display();
++a;
```

```
cout << "After ++a: ";a.display();  
--b;  
cout << "After --b: ";b.display();  
c = a + 8;  
cout << "After c = a + 8: ";c.display();  
c = 8 + a;  
cout << "After c = 8 + a: ";c.display();  
return 0;}
```

Output:

Object a: three_d(x: 1, y: 2, z: 3)
Object b: three_d(x: 4, y: 5, z: 6)
After c = a + b: three_d(x: 5, y: 7, z: 9)
After c = a - b: three_d(x: -3, y: -3, z: -3)
After ++a: three_d(x: 2, y: 3, z: 4)
After --b: three_d(x: 3, y: 4, z: 5)
After c = a + 8: three_d(x: 10, y: 11, z: 12)
After c = 8 + a: three_d(x: 10, y: 11, z: 12)